

COMPANION
to

ChatGPT Teaches TikZ

Chapter 20: Styles with Parameters (A)

Contents

20.1	What This Companion Adds	2
20.2	A Parameter Vocabulary	3
20.3	When a Parameter Is Appropriate	3
20.4	One Parameter	4
20.5	Keeping a Base Style	4
20.6	Supplying a Default	5
20.7	Two Parameters with a Delimiter	5
20.8	Choosing the Calling Syntax	6
20.9	Worked Project: One Family of Vertices	6
20.10	Debugging Workshop	8
20.11	Exercises	8
20.12	Solutions	9
20.13	ChatGPT Workshop	10
20.14	Final Checklist	10

20.1 What This Companion Adds

Chapter 20 introduced parameterized styles. This Companion develops a decision process: isolate the genuinely variable property, keep the common design in a base style, choose a readable calling syntax, and provide a default only when one value is truly standard.

Design

A parameter should expose a meaningful variation, not every low-level TikZ option that could possibly change.

20.2 A Parameter Vocabulary

Form	Purpose
<code>#1</code>	Refers to the first supplied argument.
<code>#2</code>	Refers to the second supplied argument.
<code>name=value</code>	Supplies one argument to a style.
<code>/.default=value</code>	Supplies the common value when none is written.
<code>/.style args=</code>	Defines the delimiter pattern for several arguments.
<code>{value 1/value 2}</code>	Groups two delimited arguments at the point of use.
<code>fill=#1</code>	Uses an argument as a fill color.
<code>minimum size=#2</code>	Uses an argument as a dimension.

20.3 When a Parameter Is Appropriate

Suppose four vertex styles repeat every option except the fill color. That is a family with one controlled variation. A single parameterized style is clearer than four copied definitions.

Do not parameterize a property that changes only once. A local option is simpler for an isolated exception.

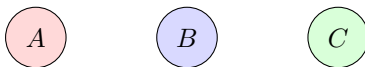
20.4 One Parameter

```
\tikzset{
  colored vertex/.style={
    draw,circle,minimum size=8mm,fill=#1
  }
}
```

The value after the equals sign replaces #1.

```
\node[colored vertex=red!15] at (0,0) {$A$};
\node[colored vertex=blue!15] at (2,0) {$B$};
\node[colored vertex=green!15] at (4,0) {$C$};
```

See



20.5 Keeping a Base Style

Put the common semantic role in a fixed style and the variation in a specialization.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  highlighted vertex/.style={
    vertex,very thick,fill=#1
  }
}
```

The parameter changes only the highlight color. Shape and size remain governed by **vertex**.

20.6 Supplying a Default

```
\tikzset{
  highlighted vertex/.style={
    vertex,very thick,fill=#1
  },
  highlighted vertex/.default=yellow!25
}
```

Both forms are now valid.

```
\node[highlighted vertex] at (0,0) {$A$};
\node[highlighted vertex=blue!15] at (2,0) {$B$};
```

The first use receives the default; the second supplies an explicit value.

Common Mistake

Do not invent a default when no value is ordinary. An arbitrary default can hide an important distinction in the source.

20.7 Two Parameters with a Delimiter

```
\tikzset{
  variable vertex/.style args={#1/#2}{
    draw,circle,fill=#1,minimum size=#2
  }
}
```

The slash is a delimiter selected by the style designer.

```
\node[variable vertex={red!15/7mm}]
  at (0,0) {$A$};
\node[variable vertex={blue!15/11mm}]
  at (2.5,0) {$B$};
```

Braces keep the complete two-part argument together inside the option list.

20.8 Choosing the Calling Syntax

Use a short pattern that readers can remember. Color/size is reasonable when a style has exactly those two variations. If a component has three or more independent settings, named keys are usually clearer than positional arguments.

20.9 Worked Project: One Family of Vertices

20.9.1 Stage 1: Define the Base and Specialization

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  highlight vertex/.style={
    vertex,very thick,fill=#1
  },
  highlight vertex/.default=yellow!25,
  edge/.style={thick}
}
```

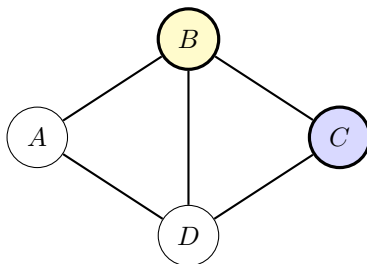
20.9.2 Stage 2: Use Default and Explicit Values

```
\node[vertex] (A) at (0,0) {$A$};
\node[highlight vertex] (B) at (2,1.3) {$B$};
\node[highlight vertex=blue!15] (C) at (4,0) {$C$};
\node[vertex] (D) at (2,-1.3) {$D$};
```

20.9.3 The Complete Picture

```
\begin{tikzpicture}[
  vertex/.style={draw,circle,minimum size=8mm},
  highlight vertex/.style={
    vertex,very thick,fill=#1
  },
  highlight vertex/.default=yellow!25,
  edge/.style={thick}
]
  \node[vertex] (A) at (0,0) {$A$};
  \node[highlight vertex] (B) at (2,1.3) {$B$};
  \node[highlight vertex=blue!15] (C) at (4,0) {$C$};
  \node[vertex] (D) at (2,-1.3) {$D$};
  \draw[edge] (A)--(B)--(C)--(D)--(A);
  \draw[edge] (B)--(D);
\end{tikzpicture}
```

See



Design

The base style controls the common design. The parameter controls only the allowed variation in highlighting.

20.10 Debugging Workshop

20.10.1 Every Node Uses the Same Value

Check that the changing option refers to `#1` rather than a fixed value.

20.10.2 A Two-Argument Style Is Unknown

Use `/.style args,` reproduce the delimiter pattern exactly, and group the supplied argument with braces.

20.10.3 Using the Style Without a Value Fails

Either supply a value explicitly or define a meaningful `/.default`.

20.10.4 The Interface Has Too Many Parameters

Keep only genuine variations. Replace several independent positional arguments with named keys in the next chapter.

20.11 Exercises

1. Define a node style whose fill is one parameter.
2. Define a useful default for that fill.
3. Parameterize the line width of an edge style.
4. Build a highlighted style from a fixed base style.
5. Define a color/size style with two arguments.

6. Change the delimiter in a two-argument style.
7. Refactor three nearly identical styles into one family.
8. Identify a component that should use named keys instead.

20.12 Solutions

20.12.1 Exercises 1–4

```
\tikzset{
  colored node/.style={draw,fill=#1},
  colored node/.default=gray!15,
  weighted edge/.style={line width=#1},
  important/.style={vertex,very thick,fill=#1}
}
```

20.12.2 Exercises 5–7

```
\tikzset{
  variable node/.style args={#1/#2}{
    draw,circle,fill=#1,minimum size=#2
  }
}
```

Exercise 6 changes both the pattern and every use consistently. Exercise 7 places the common properties once and turns only the true difference into a parameter.

For Exercise 8, prefer named keys when readers would otherwise have to remember several argument positions.

20.13 ChatGPT Workshop

Ask ChatGPT to distinguish common design from controlled variation.

1. Refactor these nearly identical styles into one parameterized family.
2. Explain exactly where **#1** and **#2** are substituted.
3. Propose a useful default and justify why it represents the ordinary case.
4. Diagnose this failing `/.style args` call and its delimiter.
5. Decide whether these variations need parameters, local options, or keys.

20.14 Final Checklist

Before continuing to Chapter 21, check that you can

1. identify a genuine style variation;
2. use **#1** in a one-parameter style;
3. build the variable style from a fixed base;
4. define and use a meaningful default;
5. define two parameters with `/.style args`;
6. choose and reproduce a readable delimiter; and
7. recognize when named keys would be clearer.